

AGA CoStays

Microservice Architecture Document

Backend Design for Multi-Branch Hotel + Restaurant Management System

Generated On: 2026-07-09 05:29:54

This document explains the backend microservice architecture for AGA CoStays clearly for implementation. It includes each microservice with controllers, services, repositories, entities, DTOs, exception handling, security, inter-service communication, important APIs and internal flow.

1. Architecture Overview

```
React Frontend
  |
  v
API Gateway
  |
  v
JWT Authentication + Role / Branch Authorization
  |
  v
Microservices
  |
  v
Database per Service
  |
  v
Async Events using Kafka / RabbitMQ
```

- Use API Gateway as the single backend entry point.
- Use JWT tokens for authentication.
- Use role-based access for ROOT_ADMIN, MANAGER, CUSTOMER, RECEPTIONIST, RESTAURANT_ADMIN, CHEF, SERVING_STAFF and HOUSEKEEPING_STAFF.
- Use branch-level validation for all staff except ROOT_ADMIN and MANAGER.
- Use database-per-service to keep microservices independent.
- Use synchronous calls for immediate validations and asynchronous events for notifications, analytics and background workflows.
- Old ADMIN role is replaced by MANAGER. Use /api/manager routes for business admin operations.

2. Role-Based Security Rules

ROOT_ADMIN:

Full access to all platform modules, all branches, website settings, audit logs and backups.

MANAGER:

Business operations access for assigned/all branches depending on accessLevel.

Can manage staff, branches, rooms, pricing, bookings, restaurant, payroll and reports.

RECEPTIONIST:

Can access only assigned branch bookings, rooms, support, attendance and branch reports.

RESTAURANT_ADMIN:

Can access only assigned branch restaurant menu, dish photos, food orders and restaurant reports.

CHEF:

Can access only assigned branch kitchen orders.

SERVING_STAFF:

Can access only assigned branch delivery orders.

HOUSEKEEPING_STAFF:

Can access only assigned branch cleaning tasks.

CUSTOMER:

Can browse public hotel data without login.

Must login for booking, food order, payment, support request, feedback and own data.

3. Microservice Summary

No.	Microservice	Main Responsibility
1	API Gateway Service	Routing, JWT filtering, CORS, rate limiting, gateway-level error handling
2	Auth Service	Registration, login, JWT, refresh token, password reset
3	User & Role Service	Root Admin, Manager, staff, customers, roles, permissions, branch mapping
4	Branch / Location Service	Cities, hotel branches, branch photos, contacts
5	Room Service	Rooms, room photos, room status, availability, location-based pricing
6	Booking Service	Booking, approval, cancellation, check-in, checkout
7	Restaurant Service	Restaurant menu, dish photos, food orders, chef and serving workflow
8	Payment Service	Payment creation, verification, refund, webhook
9	Billing Service	Final bill, invoice, room + restaurant charges
10	Notification Service	Emails, SMS, reminders, notification logs
11	Attendance Service	Staff check-in/out, absent, monthly attendance
12	Payroll Service	Salary structure, bank accounts, payroll, salary credit, salary slips
13	Analytics / Report Service	Hotel, restaurant, branch, root analytics and exports
14	Support / Feedback Service	Customer support requests and feedback

4. Common Package Structure Per Microservice

```
com.agacostays.<service-name>
|
|— controller
|— service
|   |— impl
|— repository
|— entity
|— dto
|   |— request
|   |— response
|— mapper
|— exception
|— security
|— config
|— client
|— event
|— util
|— constants
|— <ServiceName>Application.java
```

5. Common Exception Handling Standard

- Every microservice must have a GlobalExceptionHandler.
- Use consistent error response format across all services.
- Use traceId for debugging distributed requests.
- Never expose internal stack traces to frontend users.

Standard Error Response:

```
{
  "timestamp": "2026-07-09T10:30:00",
  "status": 404,
  "error": "NOT_FOUND",
  "message": "Room not found with id 2",
  "path": "/api/hotel-branches/1/rooms/2",
  "traceId": "abc-123"
}
```

Common Exceptions:

ResourceNotFoundException
BadRequestException
UnauthorizedException
AccessDeniedException
ValidationException
DuplicateResourceException
InvalidStatusException
BusinessRuleException
BranchAccessDeniedException
InvalidRoleException
UserAlreadyExistsException
InvalidCredentialsException
TokenExpiredException
FileUploadException
PaymentFailedException
BookingNotAllowedException
RoomNotAvailableException
RoomPriceUpdateException
NotificationFailedException
PayrollGenerationException
SalaryPaymentFailedException
AttendanceException
RestaurantOrderException

6. Common Security Components

JwtAuthenticationFilter
JwtTokenProvider
CustomUserDetailsService
SecurityConfig
RoleBasedAccessService
BranchAccessValidator
PermissionValidator
CurrentUserProvider
AuthenticationEntryPoint
AccessDeniedHandler
PasswordEncoderConfig
CorsConfig
MethodSecurityConfig

1. API Gateway Service

Purpose: Acts as the single entry point for frontend requests and routes traffic to the correct microservice.

Database: No database required unless storing gateway logs/rate limit state.

Security: Validates JWT at gateway level for protected routes and forwards user claims to downstream services.

Controllers

Controller	Responsibility
HealthController	Optional health/status endpoint for gateway

Services

Service	Responsibility
RouteService	Maintains route metadata if dynamic routing is used
GatewayAuthService	Validates token and extracts claims

Repositories

Repository	Entity / Data
None	Gateway should avoid business persistence

Entities / Tables

Entity	Main Fields
None	No business entities

DTOs

DTO	Purpose
GatewayErrorResponse	Standard gateway error response
RouteInfoResponse	Optional route information

Exceptions

Exception	When To Throw
GatewayRouteException	When downstream route is unavailable
GatewayUnauthorizedException	When token is missing/invalid
RateLimitExceededException	When too many requests are made

Security / Config Classes

Class	Purpose
JwtGatewayFilter	Validate JWT before routing
CorsConfig	Configure allowed frontend origins
RateLimitConfig	Rate limiting configuration
GatewayExceptionHandler	Gateway-level exception handling

Important APIs

Method	Endpoint	Access	Purpose
GET	/actuator/health	Public/Internal	Health check

Internal Flow

```
Frontend Request
-> API Gateway
-> Check route
-> Validate JWT if protected
-> Add userId, role, branchId headers
-> Route to target service
-> Return response
```

Important Notes

- Do not put business logic in API Gateway.
- Use it for routing, cross-cutting security, logging and rate limiting only.

2. Auth Service

Purpose: Handles customer registration, login, logout, JWT generation, refresh tokens and password reset.

Database: auth_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
AuthController	Register, login and logout
PasswordController	Forgot/reset/change password
TokenController	Refresh and validate tokens

Services

Service	Responsibility
AuthService	Registration and login business logic
JwtService	Generate and validate JWT tokens
PasswordService	Password reset/change logic
RefreshTokenService	Refresh token lifecycle
OtpService	Optional OTP verification

Repositories

Repository	Entity / Data
UserRepository	User entity
RoleRepository	Role entity
RefreshTokenRepository	RefreshToken entity
PasswordResetTokenRepository	PasswordResetToken entity

Entities / Tables

Entity	Main Fields
User	user_id, full_name, email, phone, password_hash, role_id, status
Role	role_id, role_name, description
RefreshToken	token_id, user_id, token, expiry_date, revoked
PasswordResetToken	reset_id, user_id, token, expiry_date, used

DTOs

DTO	Purpose
RegisterRequest	Customer registration data
LoginRequest	Email/password login
AuthResponse	JWT token and role response
ForgotPasswordRequest	Email for reset link
ResetPasswordRequest	Token and new password
RefreshTokenRequest	Refresh token request

Exceptions

Exception	When To Throw
InvalidCredentialsException	Invalid email or password
UserAlreadyExistsException	Duplicate email/phone
TokenExpiredException	Expired JWT/reset token
InvalidTokenException	Malformed or revoked token

Security / Config Classes

Class	Purpose
JwtTokenProvider	Creates and validates tokens
PasswordEncoderConfig	BCrypt password hashing
AuthSecurityConfig	Auth service security rules

Inter-Service Clients

Client	Target / Purpose
UserServiceClient	Optional client to sync profile or role data

Published / Consumed Events

Event	Direction / Purpose
UserRegisteredEvent	Published after customer registration
PasswordResetRequestedEvent	Published to Notification Service

Important APIs

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Customer registration
POST	/api/auth/login	All roles	Login
POST	/api/auth/logout	All roles	Logout
POST	/api/auth/forgot-password	Public	Forgot password
POST	/api/auth/reset-password	Public	Reset password
POST	/api/auth/refresh-token	All roles	Refresh JWT

Internal Flow

Register/Login Request

- > AuthController
- > AuthService
- > UserRepository / RoleRepository
- > JwtService creates JWT
- > Return AuthResponse

3. User & Role Service

Purpose: Manages Root Admin, Manager, staff, customers, roles, permissions and staff branch mapping.

Database: user_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
RootAdminController	Root admin profile operations
ManagerController	Manager CRUD by Root Admin
StaffController	Staff CRUD by Manager
CustomerController	Customer profile and status
RoleController	Role management
PermissionController	Permission management
StaffBranchController	Staff to branch assignment
ProfileController	Current user profile

Services

Service	Responsibility
RootAdminService	Root Admin profile logic
ManagerService	Manager management
StaffService	Staff management
CustomerService	Customer management
RoleService	Roles
PermissionService	Permissions
StaffBranchService	Branch mapping
ProfileService	Common profile operations

Repositories

Repository	Entity / Data
UserRepository	User
RootAdminProfileRepository	RootAdminProfile
ManagerProfileRepository	ManagerProfile
StaffRepository	Staff
CustomerRepository	Customer
RoleRepository	Role
PermissionRepository	Permission
RolePermissionRepository	RolePermission
StaffBranchMappingRepository	StaffBranchMapping

Entities / Tables

Entity	Main Fields
User	user_id, email, phone, password_hash, role_id, user_type, status
RootAdminProfile	root_admin_id, user_id, full_name, email, phone
ManagerProfile	manager_id, user_id, access_level, status
Staff	staff_id, user_id, department, shift, joining_date, status
Customer	customer_id, user_id, address, status
Role	role_id, role_name
Permission	permission_id, permission_code, module_name
RolePermission	role_permission_id, role_id, permission_id
StaffBranchMapping	mapping_id, staff_id, branch_id, role_id, shift, status

DTOs

DTO	Purpose
CreateManagerRequest	Root Admin creates manager
CreateStaffRequest	Manager creates staff
AssignRoleRequest	Assign role to staff
TransferBranchRequest	Move staff to another branch
UserProfileResponse	Profile response
RoleResponse	Role details
PermissionResponse	Permission details

Exceptions

Exception	When To Throw
UserNotFoundException	User not found
RoleNotFoundException	Role missing
InvalidRoleException	Invalid role assignment

DuplicateUserException	Duplicate email/phone
BranchMappingException	Invalid branch assignment

Security / Config Classes

Class	Purpose
UserSecurityConfig	Service-level security
PermissionValidator	Checks permission codes
CurrentUserProvider	Reads logged-in user claims

Inter-Service Clients

Client	Target / Purpose
BranchServiceClient	Validates branch exists
NotificationServiceClient	Sends account creation emails

Published / Consumed Events

Event	Direction / Purpose
ManagerCreatedEvent	Published after manager creation
StaffAssignedToBranchEvent	Published after assignment

Important APIs

Method	Endpoint	Access	Purpose
POST	/api/root-admin/managers	ROOT_ADMIN	Create Manager
POST	/api/manager/staff	MANAGER	Create staff
GET	/api/manager/customers	MANAGER	View customers
POST	/api/manager/hotel-branches/{branchId}/staff	MANAGER	Assign staff to branch
GET	/api/staff/my-branch	Branch Staff	Get assigned branch

Internal Flow

```
Create Staff
-> StaffController
-> StaffService
-> Validate role and branch
-> Create User + Staff
-> Create StaffBranchMapping
-> Publish StaffAssignedToBranchEvent
```

4. Branch / Location Service

Purpose: Manages cities, hotel branches, branch photos, receptionist contacts and emergency contacts.

Database: branch_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
CityController	City APIs
HotelBranchController	Branch CRUD and public branch browsing
HotelBranchPhotoController	Hotel branch photos
ReceptionistContactController	Receptionist contacts
EmergencyContactController	Emergency contacts

Services

Service	Responsibility
CityService	City logic
HotelBranchService	Branch logic
HotelBranchPhotoService	Branch photo upload/manage
ReceptionistContactService	Receptionist contacts
EmergencyContactService	Emergency contacts

Repositories

Repository	Entity / Data
CityRepository	City
HotelBranchRepository	HotelBranch
HotelBranchPhotoRepository	HotelBranchPhoto
ReceptionistContactRepository	ReceptionistContact
EmergencyContactRepository	EmergencyContact

Entities / Tables

Entity	Main Fields
City	city_id, city_name, state, country, status
HotelBranch	branch_id, city_id, branch_name, address, latitude, longitude
HotelBranchPhoto	photo_id, branch_id, photo_url, caption, is_primary
ReceptionistContact	contact_id, branch_id, staff_id, phone, shift
EmergencyContact	emergency_id, branch_id, phone, email, available

DTOs

DTO	Purpose
CreateCityRequest	Create city
HotelBranchRequest	Create/update branch
HotelBranchResponse	Branch details
BranchPhotoResponse	Branch photo details
ReceptionistContactResponse	Receptionist contact
EmergencyContactResponse	Emergency contact

Exceptions

Exception	When To Throw
CityNotFoundException	City does not exist
BranchNotFoundException	Branch missing
BranchPhotoException	Photo upload/delete error
InvalidLocationException	Invalid latitude/longitude

Security / Config Classes

Class	Purpose
BranchAccessValidator	Validates branch access
FileUploadSecurityValidator	Validates photo upload
BranchSecurityConfig	Security rules

Inter-Service Clients

Client	Target / Purpose
UserServiceClient	Resolve receptionist/staff contact details

Published / Consumed Events

Event	Direction / Purpose
BranchCreatedEvent	Published after branch creation
BranchPhotoUpdatedEvent	Published after photo change

Important APIs

Method	Endpoint	Access	Purpose
GET	/api/cities	Public	Get cities
GET	/api/hotel-branches/city/{cityId}	Public	Branches by city
GET	/api/hotel-branches/{branchId}	Public	Branch details
POST	/api/manager/hotel-branches	MANAGER	Create branch
GET	/api/hotel-branches/{branchId}/receptionists	Public	Receptionist contacts

Internal Flow

Customer Branch Browsing

- > HotelBranchController
- > HotelBranchService
- > HotelBranchRepository + PhotoRepository
- > Return branch details with photos and contacts

5. Room Service

Purpose: Manages branch rooms, room photos, room status, room availability and location-based room pricing.

Database: room_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
RoomController	Room CRUD and public list
RoomPhotoController	Room photo management
RoomPriceController	Location-based price updates
RoomAvailabilityController	Availability checks
RoomStatusController	Room status updates

Services

Service	Responsibility
RoomService	Room logic
RoomPhotoService	Photo upload/manage
RoomPriceService	Price update and history
RoomAvailabilityService	Availability validation
RoomStatusService	Status transitions

Repositories

Repository	Entity / Data
RoomRepository	Room
RoomPhotoRepository	RoomPhoto
RoomPriceHistoryRepository	RoomPriceHistory

Entities / Tables

Entity	Main Fields
Room	room_id, branch_id, room_number, room_type, base_price_per_day, current_price_per_day, status
RoomPhoto	photo_id, branch_id, room_id, photo_url, is_primary
RoomPriceHistory	price_history_id, room_id, branch_id, old_price, new_price, changed_by

DTOs

DTO	Purpose
CreateRoomRequest	Create branch room
UpdateRoomRequest	Update room details
RoomResponse	Room details with price/photos
UpdateRoomPriceRequest	Update location price
RoomPriceHistoryResponse	Price history
RoomAvailabilityResponse	Available rooms

Exceptions

Exception	When To Throw
RoomNotFoundException	Room missing
RoomNotAvailableException	Room unavailable
RoomPriceUpdateException	Invalid price update
InvalidRoomStatusException	Invalid status transition
RoomPhotoException	Photo upload/delete issue

Security / Config Classes

Class	Purpose
BranchAccessValidator	Checks branch access
RoomSecurityConfig	Room API security
FileUploadSecurityValidator	Photo validation

Inter-Service Clients

Client	Target / Purpose
BookingServiceClient	Optional check active bookings before delete/status changes
BranchServiceClient	Validate branch exists

Published / Consumed Events

Event	Direction / Purpose
RoomPriceUpdatedEvent	Published after price update
RoomStatusChangedEvent	Published after status update

Important APIs

Method	Endpoint	Access	Purpose
GET	/api/hotel-branches/{branchId}/rooms	Public	Rooms by branch
POST	/api/hotel-branches/{branchId}/rooms	RECEPTIONIST/MANAGER	Add room
PUT	/api/manager/hotel-branches/{branchId}/rooms/{roomId}/price	MANAGER	Update price
GET	/api/manager/hotel-branches/{branchId}/rooms/{roomId}/price-history	MANAGER	Price history
POST	/api/hotel-branches/{branchId}/rooms/{roomId}/photos	RECEPTIONIST/MANAGER	Upload room photos

Internal Flow

```
Update Room Price
-> RoomPriceController
-> RoomPriceService
-> Validate MANAGER/ROOT_ADMIN access
-> Fetch Room
-> Save old price in RoomPriceHistory
-> Update current_price_per_day
-> Publish RoomPriceUpdatedEvent
```

Important Notes

- Booking Service must store price_per_day_at_booking to prevent future price changes affecting old bookings.

6. Booking Service

Purpose: Handles room bookings, booking approval/rejection, cancellation, check-in, checkout and booking workflow events.

Database: booking_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
BookingController	Create and manage bookings
CustomerBookingController	Customer-owned bookings
CheckInController	Check-in
CheckOutController	Checkout
BookingStatusController	Approve/reject/cancel

Services

Service	Responsibility
BookingService	Main booking logic
BookingValidationService	Validates customer, branch and room
BookingPriceService	Calculates price using currentPricePerDay
CheckInService	Check-in workflow
CheckOutService	Checkout workflow
BookingNotificationService	Triggers booking and checkout notifications

Repositories

Repository	Entity / Data
BookingRepository	Booking
BookingHistoryRepository	BookingHistory

Entities / Tables

Entity	Main Fields
Booking	booking_id, branch_id, customer_id, room_id, dates, price_per_day_at_booking, room_charges, status
BookingHistory	history_id, booking_id, old_status, new_status, changed_by

DTOs

DTO	Purpose
CreateBookingRequest	Booking form
BookingResponse	Booking details
BookingStatusRequest	Approve/reject/cancel
CheckInRequest	ID proof details
CheckOutResponse	Checkout result

Exceptions

Exception	When To Throw
BookingNotFoundException	Booking missing
RoomNotAvailableException	Room unavailable
BookingNotAllowedException	Invalid booking action
InvalidBookingStatusException	Invalid status transition
CustomerMismatchException	Customer does not own booking

Security / Config Classes

Class	Purpose
BookingSecurityConfig	Booking security
OwnerAccessValidator	Customer ownership validation
BranchAccessValidator	Branch access validation

Inter-Service Clients

Client	Target / Purpose
RoomServiceClient	Get room/current price and validate availability
UserServiceClient	Validate customer
BillingServiceClient	Generate bill on checkout
NotificationServiceClient	Send emails

Published / Consumed Events

Event	Direction / Purpose
BookingCreatedEvent	Published after booking
BookingApprovedEvent	Published after approval
BookingCancelledEvent	Published after cancellation
CheckoutCompletedEvent	Published after checkout

Important APIs

Method	Endpoint	Access	Purpose
POST	/api/hotel-branches/{branchId}/bookings	CUSTOMER	Create booking
GET	/api/bookings/my-bookings	CUSTOMER	My bookings
PUT	/api/hotel-branches/{branchId}/bookings/{bookingId}/approve	RECEPTIONIST/MANAGER	Approve
PUT	/api/hotel-branches/{branchId}/bookings/{bookingId}/check-in	RECEPTIONIST/MANAGER	Check-in
PUT	/api/hotel-branches/{branchId}/bookings/{bookingId}/check-out	RECEPTIONIST/MANAGER	Checkout

Internal Flow

Create Booking

- > BookingController
- > BookingValidationService
- > RoomServiceClient gets current price
- > BookingPriceService calculates charges
- > BookingRepository saves booking with price_per_day_at_booking
- > Publish BookingCreatedEvent
- > Notification Service sends email

7. Restaurant Service

Purpose: Manages branch restaurants, menu, dish photos, food orders, chef workflow, serving staff workflow and restaurant feedback.

Database: restaurant_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
RestaurantController	Restaurant details
MenuController	Menu CRUD
DishPhotoController	Dish photos
FoodOrderController	Food orders
KitchenController	Chef workflow
ServingController	Serving staff workflow
RestaurantFeedbackController	Feedback
RestaurantAssignmentController	Assign chef/serving staff

Services

Service	Responsibility
RestaurantService	Restaurant details
MenuService	Menu logic
DishPhotoService	Photo logic
FoodOrderService	Order placement and status
KitchenService	Chef actions
ServingService	Delivery actions
RestaurantFeedbackService	Feedback
RestaurantOrderAssignmentService	Assign staff

Repositories

Repository	Entity / Data
RestaurantRepository	Restaurant
RestaurantMenuRepository	RestaurantMenu
RestaurantMenuPhotoRepository	RestaurantMenuPhoto
RestaurantOrderRepository	RestaurantOrder
RestaurantOrderItemRepository	RestaurantOrderItem
RestaurantFeedbackRepository	RestaurantFeedback

Entities / Tables

Entity	Main Fields
Restaurant	restaurant_id, branch_id, name, opening_time, status
RestaurantMenu	menu_item_id, branch_id, restaurant_id, name, price, availability
RestaurantMenuPhoto	photo_id, branch_id, menu_item_id, photo_url
RestaurantOrder	order_id, branch_id, booking_id, customer_id, status, amount
RestaurantOrderItem	order_item_id, order_id, menu_item_id, quantity, price
RestaurantFeedback	feedback_id, order_id, ratings, comments

DTOs

DTO	Purpose
CreateMenuItemRequest	Add dish
MenuItemResponse	Dish details
DishPhotoResponse	Dish photo
CreateFoodOrderRequest	Food order
FoodOrderResponse	Order details
AssignChefRequest	Assign chef
AssignServingStaffRequest	Assign delivery staff
RestaurantFeedbackRequest	Feedback

Exceptions

Exception	When To Throw
MenuItemNotFoundException	Dish missing
FoodOrderNotFoundException	Order missing
InvalidFoodOrderStatusException	Invalid status update
DishUnavailableException	Dish unavailable
StaffBranchMismatchException	Assigned staff not in same branch

Security / Config Classes

Class	Purpose
RestaurantSecurityConfig	Restaurant security
BranchAccessValidator	Branch validation
OrderOwnerValidator	Customer order ownership

Inter-Service Clients

Client	Target / Purpose
BookingServiceClient	Validate booking and branch
UserServiceClient	Validate chef/serving staff branch
PaymentServiceClient	Payment status
BillingServiceClient	Add food to final bill

Published / Consumed Events

Event	Direction / Purpose
FoodOrderPlacedEvent	Published after order placed
FoodOrderAcceptedEvent	Published after accepted
FoodReadyEvent	Published when chef marks ready
FoodDeliveredEvent	Published after delivery

Important APIs

Method	Endpoint	Access	Purpose
GET	/api/hotel-branches/{branchId}/restaurant/menu	Public	Menu
POST	/api/hotel-branches/{branchId}/restaurant/menu	RESTAURANT_ADMIN/MANAGER	Add dish
POST	/api/hotel-branches/{branchId}/restaurant/orders	CUSTOMER	Place order
PUT	/api/staff/my-branch/restaurant/kitchen/orders/{orderId}/start-preparing	CHEF	Start preparing
PUT	/api/staff/my-branch/restaurant/serving/orders/{orderId}/delivered	SERVING_STAFF	Delivered

Internal Flow

Food Order

- > FoodOrderController
- > Validate customer booking and branch
- > Save order as PLACED
- > Receptionist/Restaurant Admin accepts
- > Chef prepares and marks ready
- > Serving staff delivers
- > Publish FoodDeliveredEvent

8. Payment Service

Purpose: Handles customer payments for room booking, restaurant order, final bill, refunds and payment gateway webhooks.

Database: payment_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
PaymentController	Create and verify payments
PaymentWebhookController	Gateway callbacks
RefundController	Refunds
RestaurantPaymentController	Food payment endpoints

Services

Service	Responsibility
PaymentService	Payment lifecycle
PaymentGatewayService	External gateway integration
PaymentVerificationService	Signature verification
RefundService	Refund workflow
PaymentWebhookService	Webhook processing

Repositories

Repository	Entity / Data
PaymentRepository	Payment
RefundRepository	Refund
PaymentTransactionRepository	PaymentTransaction

Entities / Tables

Entity	Main Fields
Payment	payment_id, branch_id, payment_for, reference_id, amount, status
Refund	refund_id, payment_id, amount, reason, status
PaymentTransaction	transaction_id, payment_id, gateway_ids, status

DTOs

DTO	Purpose
CreatePaymentRequest	Create payment
PaymentResponse	Payment redirect/status
VerifyPaymentRequest	Verify gateway signature
RefundRequest	Refund request
PaymentWebhookRequest	Gateway webhook payload

Exceptions

Exception	When To Throw
PaymentFailedException	Payment failed
PaymentVerificationException	Invalid signature
RefundNotAllowedException	Refund not eligible
PaymentNotFoundException	Payment missing

Security / Config Classes

Class	Purpose
PaymentSecurityConfig	Payment security
WebhookSignatureValidator	Gateway webhook validation
OwnerAccessValidator	Payment owner validation

Inter-Service Clients

Client	Target / Purpose
BookingServiceClient	Update booking payment status
BillingServiceClient	Update bill payment
RestaurantServiceClient	Update order payment
NotificationServiceClient	Payment email

Published / Consumed Events

Event	Direction / Purpose
PaymentSuccessEvent	Published after payment success

PaymentFailedEvent	Published after failure
RefundProcessedEvent	Published after refund

Important APIs

Method	Endpoint	Access	Purpose
POST	/api/payments/create	CUSTOMER/MANAGER	Create payment
POST	/api/payments/verify	CUSTOMER/MANAGER	Verify payment
POST	/api/payments/webhook	Gateway	Webhook
POST	/api/payments/{paymentId}/refund	MANAGER	Refund
POST	/api/restaurant/payments/create	CUSTOMER/MANAGER	Food payment

Internal Flow

Payment

- > Create payment gateway order
- > Customer completes payment
- > Verify signature or receive webhook
- > Save status SUCCESS/FAILED
- > Publish PaymentSuccessEvent
- > Notify dependent services

9. Billing Service

Purpose: Generates final bills, invoices and calculates room plus restaurant charges.

Database: billing_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
BillingController	Bill operations
FinalBillController	Final bill APIs
InvoiceController	Invoice generation/download

Services

Service	Responsibility
BillingService	Bill lifecycle
FinalBillService	Final bill generation
BillCalculationService	Room + restaurant charge calculation
InvoiceService	PDF invoice generation

Repositories

Repository	Entity / Data
BillingRepository	Bill
InvoiceRepository	Invoice

Entities / Tables

Entity	Main Fields
Bill	bill_id, branch_id, booking_id, room_charges, restaurant_charges, final_amount
Invoice	invoice_id, bill_id, invoice_url, generated_at

DTOs

DTO	Purpose
FinalBillResponse	Final bill details
PayFinalBillRequest	Final bill payment
InvoiceResponse	Invoice URL
BillCalculationResponse	Charge breakup

Exceptions

Exception	When To Throw
BillNotFoundException	Bill missing
InvoiceGenerationException	PDF generation failed
InvalidBillStatusException	Invalid bill state

Security / Config Classes

Class	Purpose
BillingSecurityConfig	Bill security
OwnerAccessValidator	Customer bill ownership
BranchAccessValidator	Branch checking

Inter-Service Clients

Client	Target / Purpose
BookingServiceClient	Get booking details
RestaurantServiceClient	Get restaurant charges
PaymentServiceClient	Payment status

Published / Consumed Events

Event	Direction / Purpose
FinalBillGeneratedEvent	Published after bill creation
InvoiceGeneratedEvent	Published after invoice generated

Important APIs

Method	Endpoint	Access	Purpose
GET	/api/billing/final-bill/{bookingId}	RECEPTIONIST/MANAGER	Generate bill
GET	/api/billing/my-bill/{bookingId}	CUSTOMER	My bill

POST	/api/billing/final-bill/{bookingId}/pay	CUSTOMER	Pay bill
GET	/api/billing/final-bill/{bookingId}/invoice	CUSTOMER/MANAGER	Invoice

Internal Flow

Final Bill

- > Get booking room charges
- > Get restaurant charges
- > Apply tax/discount if any
- > Save bill
- > Generate invoice
- > Return final amount

10. Notification Service

Purpose: Sends booking confirmation, payment confirmation, checkout reminder, feedback email, food order notifications and salary credited emails.

Database: notification_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
NotificationController	Notification APIs
EmailController	Email APIs
SmsController	SMS APIs
CheckoutReminderController	Checkout reminder scheduling

Services

Service	Responsibility
NotificationService	General notification
EmailService	Email sending
SmsService	SMS sending
TemplateService	Email/SMS templates
CheckoutReminderService	Reminder scheduling
NotificationLogService	Log notifications

Repositories

Repository	Entity / Data
NotificationLogRepository	NotificationLog
CheckoutReminderRepository	CheckoutReminder
EmailTemplateRepository	EmailTemplate

Entities / Tables

Entity	Main Fields
NotificationLog	notification_id, branch_id, customer_id, type, email_status
CheckoutReminder	reminder_id, booking_id, scheduled_time, status
EmailTemplate	template_id, template_code, subject, body

DTOs

DTO	Purpose
SendEmailRequest	Generic email
BookingEmailRequest	Booking confirmation
CheckoutReminderRequest	Reminder schedule
NotificationResponse	Notification status

Exceptions

Exception	When To Throw
NotificationFailedException	Email/SMS failed
TemplateNotFoundException	Email template missing
ReminderScheduleException	Could not schedule reminder

Security / Config Classes

Class	Purpose
NotificationSecurityConfig	Restrict manual notification APIs
SystemApiKeyValidator	Protect system-only APIs

Inter-Service Clients

Client	Target / Purpose
BookingServiceClient	Get booking data
BranchServiceClient	Get branch data
UserServiceClient	Get user email/phone

Published / Consumed Events

Event	Direction / Purpose
BookingCreatedEvent	Consumed to send booking email
CheckoutCompletedEvent	Consumed to send feedback email

PaymentSuccessEvent	Consumed to send payment email
SalaryCreditedEvent	Consumed to send salary email
FoodDeliveredEvent	Consumed to notify customer

Important APIs

Method	Endpoint	Access	Purpose
POST	/api/notifications/booking-confirmation-email	SYSTEM/MANAGER	Booking email
POST	/api/notifications/checkout-reminder/schedule	SYSTEM	Schedule reminder
POST	/api/notifications/feedback-email	SYSTEM	Feedback email
GET	/api/notifications/my-notifications	CUSTOMER	My notifications

Internal Flow

BookingCreatedEvent

- > Notification Service consumes event
- > TemplateService builds email
- > EmailService sends email
- > NotificationLog saved

11. Attendance Service

Purpose: Tracks employee check-in/check-out, manual attendance, absent staff, working staff and monthly attendance reports.

Database: attendance_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
AttendanceController	Manual attendance CRUD
StaffAttendanceController	Staff check-in/out
BranchAttendanceController	Branch attendance
AttendanceReportController	Reports

Services

Service	Responsibility
AttendanceService	Attendance lifecycle
AttendanceValidationService	Duplicate/branch validation
WorkingStaffService	Working/absent logic
AttendanceReportService	Monthly reports

Repositories

Repository	Entity / Data
AttendanceRepository	Attendance

Entities / Tables

Entity	Main Fields
Attendance	attendance_id, branch_id, staff_id, date, status, check_in_time, check_out_time

DTOs

DTO	Purpose
CheckInRequest	Staff check-in
CheckOutRequest	Staff check-out
MarkAttendanceRequest	Manual attendance
AttendanceResponse	Attendance details
MonthlyAttendanceResponse	Monthly summary

Exceptions

Exception	When To Throw
AttendanceNotFoundException	Attendance missing
DuplicateCheckInException	Already checked in
InvalidAttendanceStatusException	Invalid status
AttendanceException	General attendance error

Security / Config Classes

Class	Purpose
AttendanceSecurityConfig	Attendance security
BranchAccessValidator	Branch validator
StaffSelfAccessValidator	Own attendance validation

Inter-Service Clients

Client	Target / Purpose
UserServiceClient	Validate staff and branch
PayrollServiceClient	Payroll consumes attendance data

Published / Consumed Events

Event	Direction / Purpose
AttendanceMarkedEvent	Published after attendance update

Important APIs

Method	Endpoint	Access	Purpose
POST	/api/staff/my-branch/attendance/check-in	Branch Staff	Check-in
POST	/api/staff/my-branch/attendance/check-out	Branch Staff	Check-out
GET	/api/hotel-	RECEPTIONIST/MANAGER	Working staff

	branches/{branchId}/attendance/today/working		
GET	/api/hotel-branches/{branchId}/attendance/monthly	RECEPTIONIST/MANAGER	Monthly attendance

Internal Flow

Staff Check-In

- > StaffAttendanceController
- > Validate staff branch
- > Validate no duplicate check-in
- > Save Attendance PRESENT
- > Publish AttendanceMarkedEvent

12. Payroll Service

Purpose: Handles salary structures, staff bank accounts, payroll generation, salary credit, salary slips and payroll reports.

Database: payroll_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
SalaryStructureController	Salary structures by role
StaffBankAccountController	Staff bank details
PayrollController	Payroll generation/list
SalaryCreditController	Salary payment
SalarySlipController	Salary slips
PayrollReportController	Payroll reports
BranchPayrollController	Branch payroll

Services

Service	Responsibility
SalaryStructureService	Salary config
StaffBankAccountService	Bank details
PayrollService	Payroll lifecycle
PayrollCalculationService	Calculate net salary
SalaryCreditService	Bank transfer workflow
SalarySlipService	Salary slip PDF
PayrollReportService	Reports

Repositories

Repository	Entity / Data
SalaryStructureRepository	SalaryStructure
StaffBankAccountRepository	StaffBankAccount
PayrollRepository	Payroll
PayrollPaymentBatchRepository	PayrollPaymentBatch
SalarySlipRepository	SalarySlip

Entities / Tables

Entity	Main Fields
SalaryStructure	role_id, basic_salary, hra, allowances, deductions
StaffBankAccount	staff_id, bank_name, account_number, ifsc
Payroll	payroll_id, branch_id, staff_id, month, year, net_salary, status
PayrollPaymentBatch	batch_id, branch_id, total_amount, status
SalarySlip	salary_slip_id, payroll_id, salary_slip_url

DTOs

DTO	Purpose
SalaryStructureRequest	Create/update salary
StaffBankAccountRequest	Bank details
GeneratePayrollRequest	Month/year payroll
PayrollResponse	Payroll details
CreditSalaryRequest	Salary credit
SalarySlipResponse	Slip URL

Exceptions

Exception	When To Throw
SalaryStructureNotFoundException	Salary role config missing
PayrollGenerationException	Payroll calculation failed
SalaryPaymentFailedException	Bank transfer failed
PayrollAlreadyPaidException	Cannot update paid payroll
BankAccountMissingException	Staff bank account missing

Security / Config Classes

Class	Purpose
PayrollSecurityConfig	Only MANAGER/ROOT_ADMIN
BranchAccessValidator	Branch validation for branch payroll
SensitiveDataMasker	Mask bank account data

Inter-Service Clients

Client	Target / Purpose
UserServiceClient	Get staff and roles
AttendanceServiceClient	Get monthly attendance
NotificationServiceClient	Send salary notifications

Published / Consumed Events

Event	Direction / Purpose
PayrollGeneratedEvent	Published after payroll generation
SalaryCreditedEvent	Published after salary payment

Important APIs

Method	Endpoint	Access	Purpose
POST	/api/manager/payroll/salary-structures	MANAGER	Create salary structure
POST	/api/manager/payroll/generate	MANAGER	Generate payroll
POST	/api/manager/payroll/credit-salaries	MANAGER	Credit salaries
GET	/api/manager/payroll/reports/monthly-summary	MANAGER	Payroll report

Internal Flow

```
Generate Payroll
-> PayrollController
-> Get active staff from User Service
-> Get attendance from Attendance Service
-> Get salary structure
-> Calculate gross, deductions, net salary
-> Save Payroll records
-> Return summary
```

13. Analytics / Report Service

Purpose: Provides hotel, restaurant, branch, manager and root admin analytics and exports.

Database: analytics_db or read replicas / projections

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
HotelAnalyticsController	Hotel analytics
RestaurantAnalyticsController	Restaurant analytics
BranchAnalyticsController	Branch analytics
ManagerAnalyticsController	Manager analytics
RootAdminAnalyticsController	Platform analytics
ReportExportController	PDF/Excel/CSV exports

Services

Service	Responsibility
HotelAnalyticsService	Hotel metrics
RestaurantAnalyticsService	Restaurant metrics
BranchAnalyticsService	Branch KPIs
ManagerAnalyticsService	Manager dashboard
RootAdminAnalyticsService	Platform KPIs
ReportExportService	File exports

Repositories

Repository	Entity / Data
AnalyticsQueryRepository	Read-only analytics queries
ReportRepository	Generated report metadata

Entities / Tables

Entity	Main Fields
HotelRevenueView	date, branch_id, room_revenue
RestaurantSalesView	date, branch_id, dish_sales
BranchPerformanceView	branch KPIs
StaffPerformanceView	staff performance metrics

DTOs

DTO	Purpose
HotelDashboardResponse	Hotel dashboard
RestaurantDashboardResponse	Restaurant dashboard
BranchRevenueResponse	Branch revenue
RootDashboardResponse	Root dashboard
ExportReportResponse	Export file URL

Exceptions

Exception	When To Throw
ReportGenerationException	Export failed
AnalyticsDataNotFoundException	No data
InvalidDateRangeException	Invalid date range

Security / Config Classes

Class	Purpose
AnalyticsSecurityConfig	Analytics access
BranchAccessValidator	Branch-level analytics
ReportPermissionValidator	Export/report permission

Inter-Service Clients

Client	Target / Purpose
BookingServiceClient	Booking metrics
RestaurantServiceClient	Restaurant metrics
PaymentServiceClient	Revenue data
AttendanceServiceClient	Attendance metrics
PayrollServiceClient	Payroll reports

Published / Consumed Events

Event	Direction / Purpose
BookingCreatedEvent	Consumed for analytics projection
PaymentSuccessEvent	Consumed for revenue projection
FoodOrderDeliveredEvent	Consumed for restaurant analytics
AttendanceMarkedEvent	Consumed for attendance analytics

Important APIs

Method	Endpoint	Access	Purpose
GET	/api/hotel-branches/{branchId}/analytics/hotel/dashboard	RECEPTIONIST/MANAGER	Branch hotel dashboard
GET	/api/hotel-branches/{branchId}/analytics/restaurant/dashboard	RESTAURANT_ADMIN/MANAGER	Restaurant dashboard
GET	/api/manager/analytics/branches/revenue	MANAGER	Branch revenue
GET	/api/root-admin/analytics/website	ROOT_ADMIN	Website analytics

Internal Flow

Analytics Request

- > Analytics Controller
- > Validate role + branch access
- > Query read model / projection tables
- > Build dashboard response
- > Return KPIs

14. Support / Feedback Service

Purpose: Handles customer support requests, emergency requests, hotel feedback and restaurant feedback coordination.

Database: support_db

Security: JWT validation, role-based access and branch-level validation where applicable.

Controllers

Controller	Responsibility
CustomerSupportController	Support requests
FeedbackController	Hotel feedback
RestaurantFeedbackController	Restaurant feedback
SupportAssignmentController	Assign support to receptionist/staff

Services

Service	Responsibility
CustomerSupportService	Support lifecycle
FeedbackService	Hotel feedback
RestaurantFeedbackService	Restaurant feedback
SupportAssignmentService	Assign support request

Repositories

Repository	Entity / Data
CustomerSupportRequestRepository	CustomerSupportRequest
FeedbackRepository	Feedback
RestaurantFeedbackRepository	RestaurantFeedback

Entities / Tables

Entity	Main Fields
CustomerSupportRequest	request_id, branch_id, customer_id, priority, status
Feedback	feedback_id, booking_id, ratings, comments
RestaurantFeedback	feedback_id, order_id, ratings, comments

DTOs

DTO	Purpose
CreateSupportRequest	Create support
SupportRequestResponse	Support details
FeedbackRequest	Hotel feedback
RestaurantFeedbackRequest	Food feedback
AssignSupportRequest	Assign request

Exceptions

Exception	When To Throw
SupportRequestNotFoundException	Request missing
FeedbackNotAllowedException	Feedback not allowed before checkout
DuplicateFeedbackException	Already submitted
InvalidSupportStatusException	Invalid support status

Security / Config Classes

Class	Purpose
SupportSecurityConfig	Support access
OwnerAccessValidator	Customer owns request
BranchAccessValidator	Branch staff access

Inter-Service Clients

Client	Target / Purpose
BookingServiceClient	Validate booking for feedback
RestaurantServiceClient	Validate order for feedback
NotificationServiceClient	Notify support assignment

Published / Consumed Events

Event	Direction / Purpose
SupportRequestCreatedEvent	Published after support request
FeedbackSubmittedEvent	Published after feedback

Important APIs

Method	Endpoint	Access	Purpose
POST	/api/customer-support/requests	CUSTOMER	Create support request
GET	/api/customer-support/requests/my-requests	CUSTOMER	My support requests
POST	/api/feedback	CUSTOMER	Hotel feedback
POST	/api/restaurant/feedback	CUSTOMER	Restaurant feedback

Internal Flow

Support Request

- > CustomerSupportController
- > Validate customer and branch
- > Save request
- > Assign to receptionist/branch support
- > Notify staff

Final Backend Component Count Summary

Component Counts

Component	Approx. Count	Notes
Microservices	14	Recommended complete architecture
Controllers	66-67	Across all services
Service Interfaces	~73	If using interface-based design
Service Implementations	~73	One implementation per service interface
Repositories	~48	JPA repositories or equivalent
Entities	~46	Core persistence entities
Database Views / Read Models	~4	Analytics projections
DTOs	120-160	Request, response, internal and event DTOs
Mapper Classes	40-60	Entity/DTO mapping
Exception Classes	25-35	Common + module-specific exceptions
Security Classes	25-30	JWT, role, branch and permission validation
Config Classes	25-35	Security, Swagger, Kafka, Feign, CORS, storage etc.
Feign/WebClient Clients	20-30	Inter-service sync communication
Event Classes	10-15	Kafka/RabbitMQ async events

Recommended Backend Build Order

Phase 1:
API Gateway
Auth Service
User & Role Service
Branch / Location Service
Room Service
Booking Service

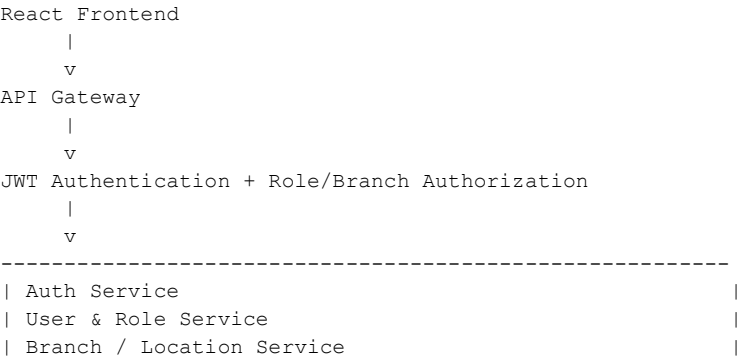
Phase 2:
Payment Service
Billing Service
Notification Service

Phase 3:
Restaurant Service
Chef Workflow
Serving Staff Workflow

Phase 4:
Attendance Service
Payroll Service

Phase 5:
Analytics / Report Service
Support / Feedback Service
Audit Logs
Exports
Backups

Final Architecture Diagram



Room Service	
Booking Service	
Restaurant Service	
Payment Service	
Billing Service	
Notification Service	
Attendance Service	
Payroll Service	
Analytics / Report Service	
Support / Feedback Service	

|
v
Database per Service
|
v
Kafka / RabbitMQ for Async Events